

Gemini 프롬프트 템플릿 매니저

최종 보고서

기술 세미나 발표 및 향후 프로젝트 참고자료 자산화 보고서

배포 URL

tea-time-six.vercel.app

작성일자

2026년 7월 29일

개발 프레임워크

Next.js 16 (App Router)

1. 시스템 개요 및 현재 운영 현황

1.1 시스템 개요

'Gemini 프롬프트 템플릿 매니저'는 생성형 AI(Google Gemini API) 활용 시 자주 사용되는 프롬프트 구조를 템플릿화하여 통합 관리하고, 동적 변수 입력 및 다중 소스 파일(로컬 PDF, Google Drive 파일) 첨부를 통해 고품질 프롬프트를 신속하게 생성·실행·관리할 수 있는 **엔터프라이즈급 반응형 웹 서비스**입니다.

주요 기능 특징

- 프롬프트 내 `{{변수}}` 템플릿 지정 및 모범 예시 매핑
- 프롬프트 실행 내역 저장, 인라인 수정 및 Soft/Hard Delete
- 백그라운드 메일 수신함 전송 및 429 쿼터 경고 알림 구축
- Footer 기반 전역 PC/모바일 보기 모드 토글 지원

운영 현황

- 배포 환경:** Vercel Platform (자동 CI/CD 연동)
- DB & 인증:** Supabase (PostgreSQL, Auth, RLS)
- 운영 상태:** 정상 운용 중 (관리자 계정 통합 제어 및 API 쿼터 자동 모니터링 활성화)

2. 시스템 요구사항 및 구현 내용

구분	주요 요구사항	구현 내용 및 반영 사항
인증/권한	관리자 승인 기반 사용자 관리	- 일반 사용자 직접 회원가입 비활성화 (Signups Disabled) - Admin에 의한 계정 생성, 권한 (<code>admin</code> / <code>editor</code> / <code>user</code>) 부여 - 프로필 모달을 통한 표시 이름 및 비밀번호 수정 기능 지원
프롬프트 관리	템플릿 생성, 변수 지원, 실행	- Gemini 3.6 Flash / 3.5 Flash-Lite / 3.1 Pro 등 모델 지정 - 변수 유효성 검사 및 과거 사용 이력 즉시 조회 지원 - 실행 전 최종 프롬프트 검토 및 실행 결과 웹 인라인 수정 (<code>onBlur</code>)
첨부 파일	다중 소스 파일 텍스트 추출	로컬 파일(PDF) 및 Google Drive API 연동 파일 첨부 지원
결과 활용	데이터 내보내기 & 전송	- 결과 <code>.txt</code> 다운로드 기능 - Nodemailer(SMTP) 기반 로그인 사용자 이메일 백그라운드 자동 전송
운영/모니터링	API 사용량 및 쿼터 관리	- 사용자별 API 호출 횟수 및 <code>k</code> (천) 단위 토큰 사용량 모니터링 - 크론 이력 조회 및 1년 경과 로그 자동 정리(Auto Pruning) 429 쿼터 오류 감지 시 관리자 자동 경고 메일 발송 및 관리 콘솔 제공

3. 기술 스택 및 기술별 프로젝트 적용 사항

- Next.js 16 (App Router)**: Server Components와 Server Actions를 적극 활용하여 초기 로딩 속도 최적화 및 클라이언트 번들 사이즈 최소화. `next/og` 를 활용한 1200x630px 동적 SEO 오픈그래프 썸네일 카드 자동 생성.
- TypeScript (Strict Mode)**: 프로젝트 전반에 `strict: true` 적용. `any` 타입을 엄격히 배제하고 Domain Interface와 Supabase DB 스키마 자동 추출 타입을 결합하여 100% 타입 안전성 확보.
- Supabase (`@supabase/ssr`)**: Next.js App Router 서버-클라이언트 환경 간 쿠키/세션 동기화를 위해 최신 `@supabase/ssr` 패키지 적용. DB 단 RLS(Row Level Security) 정책을 완벽 적용하여 보안 이중 강화.
- Nodemailer (SMTP)**: 프롬프트 결과 메일 발송 및 API 쿼터 초과 시 관리자 경고 메일 백그라운드 발송 Engine 구현.
- Vanilla CSS / CSS Modules & Tailwind CSS v4**: 글로벌 디자인 시스템(`:root` 토큰 변수)을 기반으로 CSS Modules와 Tailwind CSS v4를 혼용하여 고유하고 세련된 Dark/Light 디자인 구축.

4. 클린 아키텍처 (Clean Architecture) 및 설계 패턴 적용

본 프로젝트는 프레임워크 종속성을 낮추고 테스트 및 유지보수성을 극대화하기 위해 계층형 클린 아키텍처 구조를 엄격히 준수하였습니다.

```
src/
├── domain/                # [Domain Layer] 외부 기술에 독립적인 순수 비즈니스 로직
│   ├── entities/         # 데이터 엔티티 (User, PromptTemplate, QuotaLog 등)
│   ├── repositories/     # 저장소 추상 인터페이스 (ProductRepository 등)
│   └── usecases/         # 비즈니스 오케스트레이션 (ExecutePromptUseCase 등)
├── infrastructure/       # [Infrastructure Layer] 외부 기술 구현체
│   ├── api/              # Gemini API 및 Google Drive API 래퍼
│   ├── db/               # Supabase DB 연결 및 쿼리
│   └── repositories/     # domain/repositories 인터페이스의 실제 구현체
├── presentation/         # [Presentation Layer] UI 계층
│   ├── components/       # 순수 React 컴포넌트 & 원자 컴포넌트
│   ├── hooks/            # Usecase 연동 커스텀 훅
│   └── store/             # Zustand 클라이언트 전역 UI 상태
├── shared/               # 공통 유틸리티, 상수, 에러 객체
└── app/                  # [Framework Layer] Next.js App Router 라우트 (진입점)
```

💡 의존성 역전 원칙 (DIP) 적용 효과

Domain 계층의 `Repository Interface` 를 정의하고, Infrastructure 계층에서 이를 구현(`implements`)함으로써 도메인 로직이 특정 DB(Supabase)나 외부 API에 직접 의존하지 않도록 설계했습니다. 이를 통해 DB 교체나 외부 서비스 변경 시 도메인 영역 코드 수정 없이 유연한 확장과 Mocking 테스트가 가능합니다.

5. Antigravity와의 AI Pair Programming 협업 내용 및 노하우

5.1 AI 협업 방식 및 개발 워크플로우

- **규칙 기반 가이드라인 통제 (`.agents/rules/`)**: `tech-stack.md` , `architecture.md` , `AGENTS.md` 등을 명시하여 AI가 생성하는 모든 코드가 클린 아키텍처 규칙 및 TypeScript Strict 규칙을 일탈하지 않도록 엄격 제어.
- **TDD (Red-Green-Refactor) 기반 개발**: 단위/통합 테스트 케이스를 우선 작성(`system_test_cases.md`)하고, Antigravity 에이전트가 이를 통과하는 코드를 작성 및 리팩토링하도록 유도.
- **서브에이전트(Subagents) 및 특화 스킬(Skills) 활용**: 코드베이스 탐색, PRD 검증, 리팩토링 및 린트 검사를 병렬로 처리하여 순수 개발 시간 약 **60% 단축**.

5.2 AI 활용 생산성 성과

Strict 타입 안전성 100% 확보, 린트 에러 0건 유지, 요구사항 변경 시 테스트 케이스와 코드 간 자동 동기화로 품질 혁신 달성.

| 6. 주요 트러블슈팅 및 레슨 러닝 (Lessons Learned)

● Supabase SSR 세션 동기화 이슈

문제: Server Component와 Client Component 간 쿠키 세션 전달 시 인증 상태 불일치 발생.

해결: 최신 `@supabase/ssr` 기반의

`createBrowserClient` 및 `createServerClient` 를 분리 적용하고 middleware 에서 세션 자동 갱신 처리.

● Gemini API 쿼터 제한(429 Error) 대응

문제: 대용량 프롬프트 처리 시 429 Resource Exhausted 에러 발생으로 서비스 지연 위험.

해결: 429 감지 시 `quota_error_logs` DB 테이블 기록 및 Nodemailer 백그라운드 경고 메일을 관리자에게 즉시 발송하는 모니터링 구축.

| 7. 프로젝트 유지보수 및 온보딩 가이드

7.1 로컬 개발 환경 설정 및 실행

```
# 1. 패키지 설치
npm install

# 2. 환경 변수(.env) 설정 (Supabase, Gemini API, SMTP 정보)
# 3. 개발 서버 실행
npm run dev
```

7.2 데이터베이스 마이그레이션 관리

DB 스키마 변경 사항은 `supabase/migrations/` 폴더 내에 마이그레이션 SQL 파일로 관리되며, 신규 변경 시 사용자 사전 승인 절차를 준수합니다.

| 8. 아쉬운 점 및 향후 보완점 (로드맵)

8.1 아쉬운 점

대용량 PDF 문서 처리 시 서버 리소스 사용량 증대로 인한 대기 시간 및 실시간 Latency 시각화 그래프의 미비.

8.2 향후 발전 로드맵

- 스트리밍(Streaming) 응답 도입:** Gemini API 응답을 타자기(Typing) 실시간 스트리밍 효과로 수신하여 체감 대기 시간 축소.
- 프롬프트 템플릿 마켓플레이스/공유 기능:** 팀원 간 우수 프롬프트를 공개/공유하고 평가(좋아요/댓글)할 수 있는 커뮤니티 확장.
- Multi-LLM 확장:** Google Gemini 외 Anthropic Claude, OpenAI GPT-4o 등 다중 모델 비교 실행 지원.